

Angular Concepts

Complete Reference Guide

SupplyChainX Frontend Application

Every Angular concept listed below is explained with a simple definition and mapped to real code from the SupplyChainX frontend — so you can see exactly how each pattern is used in a production Angular application.

Angular 20

TypeScript

Reactive Forms

JWT Auth

HTTP Client

Route Guards

Bootstrap 5

48+

Concepts Covered

80+

Code Examples

20+

Components Analysed

15+

Services Mapped

Table of Contents

1 Angular Application Bootstrapping

Definition

Bootstrapping is the process of starting an Angular application. Angular reads the root component and configuration, then launches the app inside the <app-root> HTML element. Modern Angular (v17+) uses the standalone bootstrapApplication() function instead of NgModule.

How it works in SupplyChainX

File: src/main.ts — Entry point of the application.

```
import { bootstrapApplication } from '@angular/platform-browser';
import { registerLocaleData } from '@angular/common';
import localeEnIn from '@angular/common/locales/en-IN';
import { appConfig } from './app/app.config';
import { App } from './app/app';

registerLocaleData(localeEnIn);          // register Indian locale for pipes

bootstrapApplication(App, appConfig)    // ð launch the app
  .catch((err) => console.error(err));
```

TYPESCRIPT

File: src/app/app.config.ts — Application-level providers.

```
export const appConfig: ApplicationConfig = {
  providers: [
    { provide: LOCALE_ID, useValue: 'en-IN' },
    { provide: DEFAULT_CURRENCY_CODE, useValue: 'INR' },
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
    provideHttpClient(withInterceptors([authInterceptor, errorInterceptor]))
  ]
};
```

TYPESCRIPT

!provideHttpClient() registers the HTTP client globally. withInterceptors() attaches the auth & error interceptors automatically to every request.

2 Creating a New Angular Application

Definition

Angular CLI (Command Line Interface) scaffolds a complete project with one command.

```
# Install the CLI globally (once)
npm install -g @angular/cli

# Create a new project
ng new SupplyChainX-FrontEnd

# Choose: Would you like to add Angular routing? !' Yes
# Choose: Which stylesheet format? !' SCSS
```

SHELL

Key generated files in SupplyChainX

- a

ng
ula
r.js
on
—
wo
rks
pa
ce
co
nfi
g (
pr
oje
ct
na
me
:"
Su
ppl
yC
hai
nX
-Fr
ont
En
d",
st
yle
Ex
t: s
cs
s)
• p
ac
ka
ge.
jso
n
— n
pm
de
pe
nd
en
cie
s,
An
gul
ar
20
us
ed
he
re

- t
sc
onf
ig.j
so
n
–
Ty
pe
Sc
rip
t c
om
pil
er
opt
ion
s
- s
rc/
ma
in.t
s
– b
oot
str
ap
pin
g e
ntr
y p
oin
t
- s
rc/i
nd
ex.
ht
ml
– h
ost
H
T
ML
wi
th
<a
pp
-ro
ot
>

3

Running the Default Application

Definition

ng serve compiles the app, starts a local dev server and hot-reloads on file changes.

```
cd SupplyChainX-FrontEnd
npm start           # runs 'ng serve'
# !' http://localhost:4200
```

SHELL

In SupplyChainX — package.json scripts

```
"scripts": {
  "start": "ng serve",
  "build": "ng build",
  "watch": "ng build --watch --configuration development",
  "test": "ng test"
}
```

JSON

The API is consumed from <https://localhost:7295/api/> (defined in src/app/settings/app-settings.ts).

4

@Component Decorator

Definition

@Component is a TypeScript decorator that marks a class as an Angular component and provides metadata: selector (HTML tag), templateUrl/template (view), styleUrls/styles (CSS), imports (dependencies).

Example – LoginComponent

```
@Component({
  selector:      'app-login',          // <app-login> in HTML
  standalone:    true,                 // no NgModule needed
  imports:       [CommonModule, ReactiveFormsModule],
  templateUrl:   './login.component.html',
  styleUrls:     ['./login.component.scss']
})
export class LoginComponent implements OnInit { ... }
```

TYPESCRIPT

Example – ToastContainerComponent (inline template)


```
@Component({
  selector: 'app-toast-container',
  standalone: true,
  imports: [CommonModule],
  template: `          // !• inline template
    <div class='toast-container'>
      <div *ngFor='let toast of toasts' class='toast'>
        {{ toast.message }}
      </div>
    </div>`,
  styles: [`.toast-container { position: fixed; top: 20px; ... } `]
})
export class ToastContainerComponent { ... }
```

TYPESCRIPT

5 Selector

Definition

The selector property in @Component defines the custom HTML tag by which Angular identifies and renders the component inside templates. The prefix "app-" is the project default.

Selectors used in SupplyChainX

```
selector: 'app-root'           !' <app-root> in index.html
selector: 'app-login'         !' <app-login> in routes
selector: 'app-header'        !' <app-header> in main-layout.html
selector: 'app-sidebar'        !' <app-sidebar> in main-layout.html
selector: 'app-notification-center' !' <app-notification-center> in header.html
selector: 'app-toast-container' !' <app-toast-container> in app.html
```

TYPESCRIPT

main-layout.component.html – composing child selectors

```
<div class='layout-container'>
  <app-header></app-header>          <!-- HeaderComponent -->
  <div class='layout-content'>
    <app-sidebar></app-sidebar>      <!-- SidebarComponent -->
    <main class='main-content'>
      <router-outlet></router-outlet>
    </main>
  </div>
</div>
```

HTML

6 templateUrl vs template

Definition

templateUrl points to an external .html file (recommended for complex UI). template accepts an inline HTML string directly inside the decorator (good for simple, short components).


```
// templateUrl !' LoginComponent, HeaderComponent, SidebarComponent, ...
@Component({ templateUrl: './login.component.html' })

// template (inline) !' ToastContainerComponent
@Component({
  template: `
    <div class='toast-container'>
      <div *ngFor='let toast of toasts'>{{ toast.message }}</div>
    </div>
  `
})
```

TYPESCRIPT

' SupplyChainX uses templateUrl for all feature components and inline template only for the ToastContainerComponent (a tiny, self-contained widget).

7 Creating a New Component

Definition

ng generate component (or ng g c) scaffolds a new component with its .ts, .html, .scss, and .spec.ts files.

```
ng generate component components/login
# creates:
#   src/app/components/login/login.component.ts
#   src/app/components/login/login.component.html
#   src/app/components/login/login.component.scss
#   src/app/components/login/login.component.spec.ts
```

SHELL

Components in SupplyChainX (organised by dashboard)

- LoginComponent
- HeaderComponent
- SidebarComponent
- MainLayoutComponent
- AdminDashboard

dC
om
po
ne
nt

- A
dm
in
Cr
eat
eU
ser
Co
mp
on
ent

- A
dm
in
Ma
na
ge
Us
ers
Co
mp
on
ent

- A
dm
in
Au
dit
Lo
gs
Co
mp
on
ent

- A
dm
in
Ma
na
ge
Ne
tw
or
kC
om
po
ne
nt

- P
lan

ne
rD
as
hb
oa
rd
Co
mp
on
ent

- L
ogi
sti
cs
Da
sh
bo
ar
dC
om
po
ne
nt

- E
xe
cut
ive
Da
sh
bo
ar
dC
om
po
ne
nt

- E
xe
cut
ive
Kp
iS
um
ma
ry
Co
mp
on
ent

- E
xe
cut
ive
Kp
iTr

en
ds
Co
mp
on
ent

- W
ar
eh
ou
se
Da
sh
bo
ar
dC
om
po
ne
nt

- W
ar
eh
ou
se
Ma
na
gel
nv
ent
or
yC
om
po
ne
nt

- P
roc
ur
em
ent
Da
sh
bo
ar
dC
om
po
ne
nt

- A
dd
Or

de
rC
om
po
ne
nt
• V
ie
w
Or
de
rs
Co
mp
on
ent
• V
ie
wP
rofi
le
Co
mp
on
ent
• N
ot
Fo
un
dC
om
po
ne
nt
• N
otif
ica
tio
nC
ent
er
Co
mp
on
ent
• T
oa
st
Co
nta
ine
rC
om
po
ne

Component Constructor

Definition

The constructor is called when Angular creates the component. It is used for dependency injection (services, router, etc.). Heavy initialisation should go into `ngOnInit()`, not the constructor.

LoginComponent – injecting services via constructor

```
constructor(
  private fb:          FormBuilder,    // form helper
  private loginService: LoginService,  // API calls
  private router:      Router,         // navigation
  private route:       ActivatedRoute // query params
) {
  // Build the reactive form inside constructor
  this.loginForm = this.fb.group({
    email:    ['', [Validators.required, Validators.email]],
    password: ['', [Validators.required, Validators.minLength(6)]]
  });
}
```

TYPESCRIPT

AddOrderComponent – modern inject() function (Angular 14+)

```
export class AddOrderComponent implements OnInit, OnDestroy {
  private fb          = inject(FormBuilder);
  private orderService = inject(OrderService);
  private itemService  = inject(ItemService);
  private toastService = inject(ToastService);
  private orderContextService = inject(OrderContextService);

  constructor() { // constructor is now empty - DI via inject()
    this.orderForm = this.fb.group({ ... });
  }
}
```

TYPESCRIPT

Component Lifecycle Events

Definition

Angular calls lifecycle hook methods at specific moments in the component lifecycle: creation !' change detection !' destruction.

ngOnInit()

Called once after first `ngOnChanges`. Ideal for data loading (API calls, service calls).

ngOnChanges()

Called when `@Input()` properties change. Used in `ExecutiveKpiSummaryComponent`.

ngOnDestroy()

Called just before component is destroyed. Used to unsubscribe (`SidebarComponent`, `NotificationCenterComponent`).

HeaderComponent – ngOnInit()


```
ngOnInit(): void {  
  const currentUser = this.authService.currentUser;  
  if (currentUser) {  
    this.userName = currentUser.displayName || 'User';  
    this.userRole = currentUser.role;  
  }  
}
```

TYPESCRIPT

SidebarComponent – ngOnDestroy()

```
ngOnDestroy(): void {  
  this.destroy$.next();      // signal all takeUntil subscriptions to complete  
  this.destroy$.complete();  
}
```

TYPESCRIPT

10

Understanding Interpolation

Definition

Interpolation `{{ expression }}` embeds a component property or expression directly into the HTML template. Angular evaluates the expression and converts the result to a string.

header.component.html – display username and role

```
<p class='user-name-header'>{{ userName }}</p>
<p class='user-role-header'>{{ userRole }}</p>
```

HTML

login.component.html – dynamic welcome message

```
{{ successMessage }}
{{ error }}
```

HTML

executive-kpi-summary.component.html – pipes with interpolation

```
<p class='kpi-value'>{{ kpiData.otif | number:'1.1-1' }}%</p>
<p class='kpi-value'>{{ kpiData.fillRate | number:'1.1-1' }}%</p>
<p class='kpi-value'>{{ kpiData.inventoryTurn | number:'1.1-2' }}</p>
```

HTML

sidebar.component.html

```
<span class='nav-label'>{{ link.label }}</span>
```

HTML

11

Property Binding

Definition

Property binding `[property]="expression"` sets a DOM element property (or component `@Input`) to a component value at runtime. Square brackets indicate one-way binding from component to view.

login.component.html – conditionally set class and input type

```
<input [type]="showPassword ? 'text' : 'password'" formControlName="password" />
<input [class.is-invalid]="submitted && f['email'].errors" />
<button [disabled]="isSubmitting || loginForm.invalid">Sign In</button>
```

HTML

sidebar.component.html – property binding on routerLink and queryParams

```
<a [routerLink]="link.route"
  [queryParams]="link.queryParams"
  [class.active]="isLinkActive(link)">
  <i class='bi' [ngClass]="'bi-' + link.icon"></i>
  {{ link.label }}
</a>
```

HTML

executive-dashboard.component.html – passing data to child component


```
<app-executive-kpi-summary [kpiData]="kpiData"></app-executive-kpi-summary>
<app-executive-kpi-trends [trendsData]="trendsData"></app-executive-kpi-trends>
<app-executive-risks [riskData]="riskData"></app-executive-risks>
```

HTML

12

Class Binding – Single & Multiple Classes

Definition

`[class.name]="expr"` adds/removes a single CSS class based on a boolean expression. `[ngClass]="object"` applies multiple classes at once based on conditions.

Single class binding

```
<!-- warehouse-manage-inventory.component.html -->
<td [class]="isLowStock(item) ? 'low-stock' : 'in-stock'">
  {{ getStockStatus(item) }}
</td>

<!-- sidebar.component.html -->
<a [class.active]="isLinkActive(link)" class='nav-link'>...</a>

<!-- login.component.html -->
<input [class.is-invalid]="submitted && f['email'].errors" />
```

HTML

Multiple class binding with `[ngClass]`

```
<!-- executive-kpi-summary.component.html -->
<span [ngClass]="kpiData.otif >= 95 ? 'good' : 'warning'">
  {{ kpiData.otif >= 95 ? ' On Target' : '& Below Target' }}
</span>

<!-- sidebar.component.html -->
<i class='bi' [ngClass]='bi-' + link.icon"></i>
```

HTML

13

Style Binding

Definition

`[style.property]="value"` or `[ngStyle]="object"` sets inline CSS styles dynamically. The value is evaluated at runtime.

executive-dashboard.component.html – inline style

```
<div class='d-flex justify-content-center' style='height: 400px'>
  <div class='spinner-border text-primary' role='status'></div>
</div>

<!-- notification-center.component.html -->
<div [style.display]="isOpen ? 'block' : 'none'">...</div>
```

HTML

14

Understanding @if / @else (Angular 17+ Control Flow)

Definition

@if is Angular's built-in control-flow block (Angular 17+) that conditionally renders a block of HTML. It replaces the older *ngIf directive. @else provides the fallback branch.

login.component.html – show error / success messages

```
@if (successMessage) {
  <div class='alert alert-success'>
    {{ successMessage }}
  </div>
}

@if (error) {
  <div class='alert alert-danger'>{{ error }}</div>
}
```

HTML

executive-dashboard.component.html – loading vs content vs empty state

```
@if (isLoading) {
  <div class='spinner-border text-primary'></div>
}

@if (!isLoading) {
  @if (activeSection === 'summary') {
    <app-executive-kpi-summary [kpiData]="kpiData">
    </app-executive-kpi-summary>
  }
  @if (!kpiData && !riskData) {
    <div class='text-muted'>No data available</div>
  }
}
```

HTML

warehouse-manage-inventory.component.html

```
@if (!isLoading && inventory.length > 0) {
  <div class='inventory-table'>...</div>
}
@if (isLoading) {
  <div class='loading'>Loading inventory...</div>
}
@if (!isLoading && inventory.length === 0) {
  <div class='empty-state'>No inventory found.</div>
}
```

HTML

15

Understanding @for (Angular 17+ Control Flow)

Definition

@for iterates over an array and renders a template for each item. The track expression (similar to ngFor's trackBy) tells Angular how to identify items for efficient DOM updates.

sidebar.component.html – render nav links

```
@for (link of navLinks; track link.route) {
  <li class='nav-item'>
    <a [routerLink]="link.route" class='nav-link'>
      <span class='nav-label'>{{ link.label }}</span>
    </a>
  </li>
}
```

HTML

warehouse-manage-inventory.component.html – table rows

```
@for (item of inventory; track item.inventoryId) {
  <tr>
    <td>{{ getLocationName(item.locationId) }}</td>
    <td>{{ getItemName(item.itemId) }}</td>
    <td>{{ item.quantityOnHand }}</td>
    <td [class]="isLowStock(item) ? 'low-stock' : 'in-stock'">
      {{ getStockStatus(item) }}
    </td>
  </tr>
}
```

HTML

admin-create-user.component.html – dropdown options

```
@for (role of roleOptions; track role) {
  <option [value]="role">{{ role }}</option>
}
```

HTML

16

One-Way Binding vs Two-Way Binding

Definition

One-way binding ([] or {{ }}) flows data in one direction: component !' view (or view !' component via events). Two-way binding [(ngModel)] keeps the view and component in sync automatically (used in template-driven forms).

One-way – property binding (component !' view)

```
// Component
userName = 'Alice';

// Template
<p>{{ userName }}</p>           // interpolation - one-way
<input [value]="userName" />   // property binding - one-way
```

TYPESCRIPT

One-way – event binding (view !' component)


```
// Template
<button (click)="logout()">Logout</button>
<button (click)="togglePasswordVisibility()">Show</button>
```

HTML

Two-way binding – reactive forms (formControlName "H two-way")

```
// Reactive forms use formControlName for implicit two-way sync
// login.component.html
<input formControlName="email" />          <!-- view !" FormControl -->
<input formControlName="password" />      <!-- view !" FormControl -->
```

HTML

!SupplyChainX uses Reactive Forms (FormBuilder) throughout, so there is no explicit `[(ngModel)]` usage. Reactive forms give equivalent two-way sync through FormControl objects.

17

Creating Child Components & Passing Data with @Input

Definition

A child component is any component embedded inside a parent's template. @Input() decorates a property so the parent can pass data down to the child via property binding.

ExecutiveKpiSummaryComponent – child with @Input

```
// executive-kpi-summary.component.ts (CHILD)
import { Component, Input } from '@angular/core';
@Component({ selector: 'app-executive-kpi-summary', ... })
export class ExecutiveKpiSummaryComponent {
  @Input() kpiData: KpiReportDto | null = null; // data passed from parent
}
```

TYPESCRIPT

ExecutiveDashboardComponent – parent passes data down

```
// executive-dashboard.component.html (PARENT)
<app-executive-kpi-summary [kpiData]="kpiData">
</app-executive-kpi-summary>
<app-executive-kpi-trends [trendsData]="trendsData">
</app-executive-kpi-trends>
<app-executive-risks [riskData]="riskData">
</app-executive-risks>
```

HTML

Child renders the received data

```
// executive-kpi-summary.component.html
@if (kpiData) {
  <p>OTIF: {{ kpiData.otif | number:'1.1-1' }}%</p>
  <p>Fill Rate: {{ kpiData.fillRate | number:'1.1-1' }}%</p>
}
```

HTML

18

Handling Events

Definition

Event binding (event)="handler()" listens for DOM events (click, submit, change, keyup) and calls a method on the component class.

login.component.html – form submit and button click

```
<form [formGroup]="loginForm" (ngSubmit)="onSubmit()">
  ...
  <button (click)="togglePasswordVisibility()">Show</button>
  <button (click)="successMessage = null">x</button>
</form>
```

HTML

header.component.html – navigate to profile and logout


```
<a (click)="goToProfile()" class='action-link'>View Profile</a>  
<button (click)="logout()" class='logout-btn'>Logout</button>
```

[warehouse-manage-inventory.component.html – open form](#)

```
<button (click)="openCreateForm()">Create New Inventory</button>
<button (click)="openAdjustForm(item)">Adjust</button>

<!-- Stop propagation on modal -->
<div class='modal-overlay' (click)="closeAdjustForm()">
  <div class='modal-content' (click)="$event.stopPropagation()">
    </div>
  </div>
</div>
```

19 Integrating Bootstrap into Angular

Definition

Bootstrap is a popular CSS framework providing ready-made components and a responsive grid. In Angular it can be added via CDN ([index.html](#)) or npm ([package.json](#) + [angular.json](#)).

SupplyChainX – CDN approach in index.html

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
<link href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.0/font/bootstrap-icons.css" rel="stylesheet">
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>
```

Usage throughout the app

Grid: col-
md-6, col-
lg-3, row-
g-3 — in
executive-
kpi-sum-
mary, ad-

mi
n-c
re
ate
-us
er
• A
ler
ts:
ale
rt-
da
ng
er,
ale
rt-
su
cc
es
s
—
in l
ogi
n,
cre
ate
-us
er,
inv
ent
or
y
• B
utt
on
s:
btn
bt
n-
pri
ma
ry,
btn
-cl
os
e
—
thr
ou
gh
out
• F
or
ms
: fo

rm
-co
ntr
ol,
for
m-l
ab
el,
for
m-
sel
ect
, in
val
id-f
ee
db
ac
k
—
in
all
for
m
co
mp
on
ent
s
• B
oot
str
ap
lco
ns:
bi
bi-
pe
rso
n-c
ircl
e,
bi
bi-
bo
xe
s,
bi
bi-t
ru
ck
—
in
he
ad
er,

sid
eb
ar,
inv
ent
or
y
• S
pin
ne
r: s
pin
ne
r-b
or
de
r t
ext
-pr
im
ar
y
—
in
ex
ec
uti
ve-
da
sh
bo
ar
d

Reactive Forms

Definition

Reactive forms define the form structure in the component class using FormBuilder, FormGroup, and FormControl. The template binds to these objects via [formGroup] and formControlName. Validation rules are declared in TypeScript.

LoginComponent – creating the form

```
this.loginForm = this.fb.group({
  email:    ['', [Validators.required, Validators.email]],
  password: ['', [Validators.required, Validators.minLength(6)]]
});
```

TYPESCRIPT

AdminCreateUserComponent – multi-field form

```
this.userForm = this.fb.group({
  displayName: ['', Validators.required],
  email:       ['', [Validators.required, Validators.email]],
  password:    ['', [Validators.required, Validators.minLength(6)]],
  phoneNumber: ['', ],
  roleName:    ['', Validators.required],
  status:      ['Active', Validators.required]
});
```

TYPESCRIPT

AddOrderComponent – FormArray for order lines

```
this.orderForm = this.fb.group({
  orderType:      ['PO', Validators.required],
  partnerId:      [null],
  originLocationId: [null],
  destinationLocationId: [null],
  orderDate:      [new Date().toISOString().split('T')[0], Validators.required],
  expectedDeliveryDate: ['', this.expectedDeliveryDateValidator.bind(this)],
  lines: this.fb.array([]) // dynamic array of line items
});
```

TYPESCRIPT

Validation in Reactive Forms

Definition

Angular provides built-in validators (Validators.required, Validators.email, Validators.minLength, Validators.min). Custom validators can be functions that return null (valid) or an error object (invalid).

login.component.html – showing validation errors


```
@if (submitted && f['email'].errors) {
  <div class='invalid-feedback d-block'>
    @if (f['email'].errors['required']) {
      <small>Email is required</small>
    }
    @if (f['email'].errors['email']) {
      <small>Please enter a valid email</small>
    }
  </div>
}
```

HTML

admin-create-user.component.html – touched-based errors

```
@if (userForm.get('displayName')?.invalid && userForm.get('displayName')?.touched) {
  <div class='invalid-feedback d-block'>Display name is required.</div>
}
@if (userForm.get('password')?.invalid && userForm.get('password')?.touched) {
  <div class='invalid-feedback d-block'>Password must be at least 6 characters.</div>
}
```

HTML

AddOrderComponent – custom validator

```
private expectedDeliveryDateValidator(control: AbstractControl): ValidationErrors | null {
  if (!control.value) return null;
  const orderDate = this.orderForm?.get('orderDate')?.value;
  if (orderDate && control.value < orderDate) {
    return { invalidDate: true }; // return error object
  }
  return null; // valid
}
```

TYPESCRIPT

22

Data Extraction from Form Controls

Definition

Access form values via `formGroup.value` (whole form) or `formGroup.get("field").value` (individual control). Shorthand getter `f` returns `this.loginForm.controls`.

LoginComponent – extract and submit

```
get f() { return this.loginForm.controls; }

onSubmit(): void {
  if (this.loginForm.invalid) return;

  const loginRequest: LoginRequest = {
    email: this.f['email'].value,
    password: this.f['password'].value
  };
  this.loginService.login(loginRequest).subscribe({ ... });
}
```

TYPESCRIPT

WarehouseManageInventoryComponent – whole form value


```
loadInventory() {  
  const filters = this.filters.value; // { locationId, itemId, onlyLowStock, search }  
  this.inventoryService.listInventory(  
    filters.locationId || undefined,  
    filters.itemId || undefined,  
    filters.onlyLowStock,  
    filters.search || undefined  
  ).subscribe({ ... });  
}
```


Pipes & Built-in Pipes

Definition

A pipe transforms data in a template expression. Syntax: value | pipeName:arg1:arg2. Angular provides built-in pipes: number, currency, date, async, json, uppercase, lowercase.

executive-kpi-summary.component.html – number pipe

```
{{ kpiData.otif          | number:'1.1-1' }}%    // 94.567 !' 94.6%
{{ kpiData.fillRate      | number:'1.1-1' }}%    // 98.3%
{{ kpiData.inventoryTurn | number:'1.1-2' }}     // 4.23
{{ kpiData.delayRate     | number:'1.1-1' }}%
```

HTML

main.ts – registering locale for currency & number pipes

```
import { registerLocaleData } from '@angular/common';
import localeEnIn from '@angular/common/locales/en-IN';
registerLocaleData(localeEnIn); // enables 1 currency, IN number formats

// app.config.ts
{ provide: LOCALE_ID,          useValue: 'en-IN' },
{ provide: DEFAULT_CURRENCY_CODE, useValue: 'INR' },
```

TYPESCRIPT

¹ The currency pipe will now display ¹ (INR) automatically when used with | currency throughout the app.

Creating Services

Definition

Services are TypeScript classes that hold shared logic (API calls, state, utilities). They are created with: ng generate service services/order

Services in SupplyChainX

- L
- ogi
- nS
- er
- vic
- e
- a
- uth
- ent
- ica
- te
- us
- er,
- de
- co
- de
- J
- W
- T

- Authentication Service – store/retrieve current user, manage token

- Order Service – CRUD for orders (GET, POST, PUT)

- Inventory Service – li

st
an
d a
dju
st i
nv
ent
or
y

- It
em
Se
rvi
ce
– li
st
cat
alo
gu
e it
em
s

- L
oc
ati
on
Se
rvi
ce
– li
st
wa
re
ho
us
e l
oc
ati
on
s

- P
art
ne
rS
er
vic
e
– li
st
su
ppl
y c
hai
n p
art

ne
rs
• S
hip
me
nt
Se
rvi
ce
– d
isp
atc
h a
nd
tra
ck
shi
pm
ent
s
• T
oa
st
Se
rvi
ce
– s
ho
w
su
cc
es
s/e
rro
r n
otif
ica
tio
ns
•
Or
de
rC
ont
ext
Se
rvi
ce
– s
ha
re
or
de
r fil
ter

context between components (Behavioral Subject)

- Notification Event Service – poll back end for user notifications

- Manage User Service / User Service

ce
– a
dm
in
us
er
ma
na
ge
me
nt
• K
pi
Se
rvi
ce
/ E
xe
cut
ive
Da
sh
bo
ar
dS
er
vic
e
–
KP
I a
nd
ris
k d
ata

25

Understanding @Injectable & inject()

Definition

@Injectable marks a class as available for Angular's Dependency Injection system. providedIn: "root" makes it a singleton across the entire app. inject() is the modern function-based alternative to constructor injection.

@Injectable – AuthenticationService

```
@Injectable({  
  providedIn: 'root' // singleton – one instance for the whole app  
})  
export class AuthenticationService {  
  private currentUserSubject = new BehaviorSubject<CurrentUser | null>(null);  
  public currentUser$ = this.currentUserSubject.asObservable();  
  ...  
}
```

TYPESCRIPT

@Injectable – ToastService


```
@Injectable({ providedIn: 'root' })
export class ToastService {
  private toasts$ = new BehaviorSubject<Toast[]>([]);
  success(message: string, duration = 5000) { this.showToast(message, 'success',
duration); }
  error(message: string, duration = 5000) { this.showToast(message, 'error',
duration); }
}
```

TYPESCRIPT

inject() function – AddOrderComponent

```
export class AddOrderComponent {
  private fb = inject(FormBuilder);
  private orderService = inject(OrderService);
  private toastService = inject(ToastService);
  private orderContextService = inject(OrderContextService);
}
```

TYPESCRIPT

inject() function – NotificationCenterComponent

```
export class NotificationCenterComponent implements OnInit, OnDestroy {
  private notificationEventService = inject(NotificationEventService);
  private authService = inject(AuthenticationService);
}
```

TYPESCRIPT

26

Injecting Services into Components

Definition

Services are injected either through the constructor (classic) or using `inject()` (modern). Angular's DI system automatically provides the singleton instance.

Constructor injection – LoginComponent

```
constructor(
  private fb:          FormBuilder,
  private loginService: LoginService,
  private router:      Router,
  private route:       ActivatedRoute
) {}
```

TYPESCRIPT

Constructor injection – RoleGuard (class-based guard)

```
constructor(
  private authService: AuthenticationService,
  private router:      Router
) {}
```

TYPESCRIPT

27

Working with HTTP Client

Definition

`HttpClient` is Angular's service for making HTTP requests. It returns RxJS Observables. It is provided globally via `provideHttpClient()` in `app.config.ts`.

OrderService – GET, POST, PATCH, DELETE, PUT

```
// GET - list orders
listOrders(type?, status?, fromUtc?, toUtc?): Observable<OrderResponseDto[]> {
  let params = new HttpParams();
  if (type)   params = params.set('type', type);
  if (status) params = params.set('status', status);
  return this.http.get<OrderResponseDto[]>(this.apiUrl, { params });
}

// POST - place a new order
placeOrder(dto: OrderCreateDto): Observable<OrderResponseDto> {
  return this.http.post<OrderResponseDto>(this.apiUrl, dto);
}

// PATCH - update shipment status
updateShipment(id: number, dto): Observable<ShipmentResponseDto> {
  return this.http.patch<ShipmentResponseDto>(`${this.shipmentApiUrl}/${id}`, dto);
}
```

TYPESCRIPT

Working with DTOs (Data Transfer Objects)

Definition

DTOs are TypeScript interfaces that describe the shape of data sent to or received from the API. They provide type safety and IDE intellisense.

Models in SupplyChainX (src/app/models/)

- L

ogi

nR

eq

ue

st /

Lo

gin

Re

sp

on

se

– e

ma

il

+

pa

ss

wo

rd

/ J

W

T t

ok

en

-

Or

de

rCr

eat

eD

to /

Or

de

rR

es

po

ns

eD

to

– o

rd

er

pa

ylo

ad

& r

es
po
ns
e

- I
nv
ent
or
yP
osi
tio
nR
es
po
ns
eD
to /
In
ve
nto
ry
Ad
jus
tDt
o

– i
nv
ent
or
y d
ata

- It
em
Re
sp
on
se
Dt
o /
Lo
cat
ion
Re
sp
on
se
Dt
o /
Pa
rtn
er
Re
sp
on
se

Dt
o
• S
hip
me
nt
Re
sp
on
se
Dt
o /
Sh
ip
me
nt
Di
sp
atc
hD
to /
S
hip
me
nt
De
liv
er
yD
to
• K
pi
Re
po
rtD
to /
Ri
sk
Su
m
ma
ry
Dt
o /
Kp
iTr
en
ds
Dt
o
– e
xe
cut
ive
da

sh
bo
ar
d
• C
re
ate
Us
er
DT
O /
A
pp
Us
er
Re
sp
on
se
Dt
o
– u
ser
m
an
ag
em
ent
• N
otif
ica
tio
nD
to
– n
otif
ica
tio
n c
ent
er

login.model.ts – DTO example

```
export interface LoginRequest {  
  email: string;  
  password: string;  
}  
  
export interface LoginResponse {  
  token: string; // JWT token returned by backend  
}  
  
export type UserRole = 'Admin' | 'Planner' | 'Logistics' | 'Executive' | 'Procurement' |  
  'Warehouse';
```

TYPESCRIPT

HTTP Request / Response Interceptors

Definition

Interceptors sit in the HTTP pipeline and can modify every request (add headers) or every response (transform errors). They are functional in modern Angular (HttpInterceptorFn).

auth.interceptor.ts – attach JWT token to every request

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  if (req.url.includes('/Login')) return next(req); // skip login endpoint

  const authService = inject(AuthenticationService);
  const token = authService.getAuthToken();

  if (token) {
    const clonedReq = req.clone({
      setHeaders: { Authorization: `Bearer ${token}` }
    });
    return next(clonedReq).pipe(
      catchError(error => {
        if (error.status === 401) {
          authService.logout();
          router.navigate(['/login']);
        }
        return throwError(() => error);
      })
    );
  }
  return next(req);
};
```

TYPESCRIPT

error.interceptor.ts – transform server errors to user messages

```
export const errorInterceptor: HttpInterceptorFn = (req, next) => {
  return next(req).pipe(
    catchError((error: any) => {
      let errorMessage = 'An error occurred.';
      switch (error.status) {
        case 401: errorMessage = 'Session expired. Please login again.'; break;
        case 403: errorMessage = 'You do not have permission.'; break;
        case 404: errorMessage = 'Resource not found.'; break;
        case 500: errorMessage = 'Server error. Please try again later.'; break;
      }
      return throwError(() => ({ ...error, userMessage: errorMessage }));
    })
  );
};
```

TYPESCRIPT

Registered in app.config.ts

```
provideHttpClient(
  withInterceptors([authInterceptor, errorInterceptor])
)
```

TYPESCRIPT

30

Working with JWT Token – Extracting Claims & Adding to Header

Definition

A JWT (JSON Web Token) has three parts: header.payload.signature. The payload contains claims (user data). Angular decodes it manually using `atob()` and `JSON.parse()`.

login.service.ts – decode JWT and extract claims

```
private decodeToken(token: string): any {
  const base64Url = token.split('.')[1];           // middle segment = payload
  const base64     = base64Url.replace(/-/g, '+').replace(/_/g, '/');
  const json       = decodeURIComponent(          // URL-decode each byte
    atob(base64).split('').map(c =>
      '%' + ('00' + c.charCodeAt(0).toString(16)).slice(-2)
    ).join('')
  );
  return JSON.parse(json);
}
```

TYPESCRIPT

Extracting user claims from the decoded payload

```
const email = payload['http://schemas.xmlsoap.org/ws/.../emailaddress']
  || payload['email']
  || payload['sub'];

const role  = payload['http://schemas.microsoft.com/.../role']
  || payload['role'];

const name  = payload['http://schemas.xmlsoap.org/.../name']
  || payload['name'] || email;
```

TYPESCRIPT

Storing and retrieving token – AuthenticationService

```
setCurrentUser(user: CurrentUser, token: string): void {
  this.currentUserSubject.next(user);
  localStorage.setItem('authToken', token);      // persist across refresh
  localStorage.setItem('currentUser', JSON.stringify(user));
}

getAuthToken(): string | null {
  return localStorage.getItem('authToken');
}
```

TYPESCRIPT

Token added to header – auth.interceptor.ts

```
const clonedReq = req.clone({
  setHeaders: { Authorization: `Bearer ${token}` }
});
```

TYPESCRIPT

31

Defining Routes

Definition

Routes map URL paths to components. They are defined as an array of Route objects in `app.routes.ts` and registered via `provideRouter(routes)` in `app.config.ts`.

`app.routes.ts` – route structure

```
export const routes: Routes = [
  { path: 'login', component: LoginComponent },
  {
    path: '',
    component: MainLayoutComponent,
    canActivate: [AuthGuard], // guard: must be logged in
    children: [
      {
        path: 'admin-dashboard',
        component: AdminDashboardComponent,
        canActivate: [RoleGuard],
        data: { roles: ['Admin'] }, // only Admin role
        children: [...]
      },
      { path: 'planner-dashboard', ... },
    ]
  },
  { path: '404', component: NotFoundComponent },
  { path: '**', redirectTo: '404' } // catch-all
];
```

TYPESCRIPT

32

Redirect Route, Catch-All Route & Child Routes

Redirect route

```
// Redirect empty path to admin-dashboard
{ path: '', redirectTo: 'admin-dashboard', pathMatch: 'full' }

// Redirect logistics root to orders sub-page
{ path: '', redirectTo: 'orders', pathMatch: 'full' }
```

TYPESCRIPT

Catch-all route

```
{ path: '404', component: NotFoundComponent },
{ path: '**', redirectTo: '404' } // ** matches any unknown URL
```

TYPESCRIPT

Child routes – admin-dashboard

TYPESCRIPT

```
{
  path: 'admin-dashboard',
  component: AdminDashboardComponent,
  children: [
    { path: '', loadComponent: () =>
import('path:../AdminDashboardComponentViewProfileComponent') },
    { path: 'users', loadComponent: () =>
import('path:../AdminManageUsersComponent') }, () => import('../AdminAuditLogsComponent') },
    { path: 'create-user', loadComponent: () =>
import('path:../AdminCreateUserComponent') },
    { path: 'manage-network', loadComponent: () =>
import('../AdminManageNetworkComponent') }
  ]
}
```

loadComponent() enables lazy loading – Angular only downloads the component bundle when the route is first visited, significantly reducing initial load time.

33

Static Links & Dynamic Navigation

Static links with routerLink

HTML

```
<!-- sidebar.component.html - static links for each nav item -->
<a [routerLink]="/admin-dashboard" class='nav-link'>Dashboard</a>
<a [routerLink]="/admin-dashboard/users" class='nav-link'>Manage Users</a>
```

Dynamic navigation from TypeScript – Router.navigate()

TYPESCRIPT

```
// header.component.ts - navigate to profile based on current dashboard
goToProfile(): void {
  const currentUrl = this.router.url;
  if (currentUrl.includes('admin-dashboard')) {
    this.router.navigate(['/admin-dashboard/profile']);
  } else if (currentUrl.includes('planner-dashboard')) {
    this.router.navigate(['/planner-dashboard/profile']);
  }
}

// login.component.ts - navigate after successful login
this.router.navigate([redirectUrl], { replaceUrl: true });
```

Navigation with queryParams – sidebar executive section


```
navLinks = [  
  { label: 'KPI Trends', route: '/executive-dashboard', queryParams: { section:  
    'trends' } },  
  { label: 'Risk', route: '/executive-dashboard', queryParams: { section:  
    'risk' } },  
]  
  
// template  
<a [routerLink]="link.route" [queryParams]="link.queryParams">{{ link.label }}</a>  
  
// executive-dashboard.component.ts - read query param  
this.route.queryParams.subscribe(params => {  
  this.activeSection = params['section'] || 'summary';  
});
```


34

Navigation with Data & Passing Parameters to Routes

Passing query parameters

```
// auth.guard.ts - pass returnUrl as query param
router.navigate(['/login'], { queryParams: { returnUrl: state.url } });

// login.component.ts - read returnUrl
this.returnUrl = this.route.snapshot.queryParams['returnUrl'] || '';
```

TYPESCRIPT

Passing route data – roles for guards

```
// app.routes.ts - static data on route
{
  path: 'admin-dashboard',
  data: { roles: ['Admin'] }, // data passed to guard
  canActivate: [RoleGuard]
}

// role.guard.ts - read data from route
const requiredRoles = route.data['roles'] as UserRole[];
```

TYPESCRIPT

Resolvers – passing data to components before they activate

```
// app.routes.ts
{
  path: 'orders',
  loadComponent: () => import('./...ViewOrdersComponent'),
  resolve: { orderContext: OrderContextResolver } // run before component loads
}

// order-context.resolver.ts
export class OrderContextResolver implements Resolve<void> {
  private orderContextService = inject(OrderContextService);
  resolve(): void {
    this.orderContextService.clearDisabledOrderTypes();
    this.orderContextService.clearVisibleOrderTypes();
  }
}
```

TYPESCRIPT

35

Working with Route Guards

Definition

Route guards protect routes by running logic before navigation completes. CanActivate runs before a route activates. CanActivateChild runs before child routes activate.

authGuard (functional guard) – checks if user is logged in


```
export const authGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthenticationService);
  const router      = inject(Router);

  if (authService.isLoggedIn()) return true;

  router.navigate(['/login'], { queryParams: { returnUrl: state.url } });
  return false;
};
```

TYPESCRIPT

RoleGuard (class-based guard) – checks user role

```
@Injectable({ providedIn: 'root' })
export class RoleGuard implements CanActivate {
  canActivate(route: ActivatedRouteSnapshot, state: RouterStateSnapshot): boolean {
    const requiredRoles = route.data['roles'] as UserRole[];
    if (this.authService.hasRole(requiredRoles)) return true;

    // redirect to user's own dashboard
    const dashboard = roleDashboards[this.authService.currentRole!];
    this.router.navigate([dashboard]);
    return false;
  }
}
```

TYPESCRIPT

Applied in routes

```
{
  path: '',
  component: MainLayoutComponent,
  canActivate: [authGuard], // must be logged in
  children: [
    {
      path: 'admin-dashboard',
      canActivate: [RoleGuard],
      canActivateChild: [RoleGuard], // protects all child routes too
      data: { roles: ['Admin'] }
    }
  ]
}
```

TYPESCRIPT

36

Browser Storage

Definition

Browser provides `localStorage` (persists until cleared), `sessionStorage` (clears on tab close), and cookies. Angular apps access them directly via `window.localStorage`.

AuthenticationService – store/retrieve/clear auth data

```
// Store after login
localStorage.setItem('authToken', token);
localStorage.setItem('currentUser', JSON.stringify(user));

// Retrieve on app startup (constructor)
private getUserFromStorage(): CurrentUser | null {
  const userStr = localStorage.getItem('currentUser');
  return userStr ? JSON.parse(userStr) : null;
}

// Clear on logout
logout(): void {
  localStorage.removeItem('authToken');
  localStorage.removeItem('currentUser');
  this.currentUserSubject.next(null);
}
```

TYPESCRIPT

!The JWT token stored in `localStorage` is automatically picked up by `AuthenticationService` on app restart, keeping the user logged in across browser refreshes.

37

Communication Between Sibling Components

Definition

Sibling components cannot communicate directly. The pattern used in Angular is a Shared Service with a `BehaviorSubject` (RxJS). One sibling pushes data; the other subscribes and receives it.

OrderContextService – shared state via BehaviorSubject

```
@Injectable({ providedIn: 'root' })
export class OrderContextService {
  private disabledOrderTypesSubject = new BehaviorSubject<string[]>([]);
  disabledOrderTypes$ = this.disabledOrderTypesSubject.asObservable();

  setDisabledOrderTypes(orderTypes: string[]): void {
    this.disabledOrderTypesSubject.next(orderTypes); // PUSH
  }
}
```

TYPESCRIPT

AddOrderComponent – subscribes to shared state


```
// AddOrderComponent (sibling 1) - SUBSCRIBE
this.orderContextService.disabledOrderTypes$
  .pipe(takeUntil(this.destroy$))
  .subscribe((disabledTypes) => {
    this.disabledOrderTypes = disabledTypes;
  });
```

TYPESCRIPT

OrderContextResolver – pushes state before navigation

```
// ProcurementOrderContextResolver (sibling 2 side) - PUSH via resolver
resolve(): void {
  this.orderContextService.setDisabledOrderTypes(['SO']);
  this.orderContextService.setVisibleOrderTypes(['PO', 'Transfer']);
}
```

TYPESCRIPT

ToastService – BehaviorSubject for toast notifications

```
// ToastService (PUSH)
private toasts$ = new BehaviorSubject<Toast[]>([]);
success(message: string) { this.showToast(message, 'success'); }

// ToastContainerComponent (SUBSCRIBE)
this.toastService.getToasts().subscribe(toasts => {
  this.toasts = toasts;
});
```

TYPESCRIPT

38

Integrating HTTP Client with Services

Definition

The standard pattern: inject HttpClient into a service, call HTTP methods, return Observables. Components subscribe to these Observables to get data.

InventoryService !' WarehouseManageInventoryComponent

```
// inventory.service.ts
@Injectable({ providedIn: 'root' })
export class InventoryService {
  private apiUrl = AppSettings.apiEndpoint + 'Inventory';
  constructor(private http: HttpClient) {}

  listInventory(locationId?, itemId?, onlyLowStock?, search?) {
    let params = new HttpParams();
    if (locationId) params = params.set('locationId', locationId);
    return this.http.get<InventoryPositionResponseDto[]>(this.apiUrl, { params });
  }
}

// warehouse-manage-inventory.component.ts - inject and use
this.inventoryService.listInventory(...).subscribe({
  next: (data) => { this.inventory = data; },
  error: (err) => { console.error(err); }
});
```

TYPESCRIPT

39

Injecting Parameterized Service

Definition

Services can receive configuration values via Angular's InjectionToken, or by wrapping the service call with parameters. AppSettings acts as a centralized config injected into services.

AppSettings – centralized API endpoint

```
// app-settings.ts
export const AppSettings = {
  apiEndpoint: 'https://localhost:7295/api/',
  applicationUrl: 'https://localhost:7295'
};

// Used by every service as the base URL
private apiUrl = AppSettings.apiEndpoint + 'orders'; // OrderService
private apiUrl = AppSettings.apiEndpoint + 'Inventory'; // InventoryService
private apiUrl = AppSettings.apiEndpoint + 'User'; // LoginService
```

TYPESCRIPT

40

Working with Template-Driven Forms

Definition

Template-driven forms define validation and structure in the HTML template using directives like ngModel, required, minlength, etc. Angular creates FormControl objects implicitly from these directives.

!SupplyChainX uses Reactive Forms for all feature forms. Template-driven patterns appear in the filter forms where simple binding is sufficient, e.g., filters in warehouse-manage-inventory using formControlName (which is the reactive equivalent).

Template-driven pattern example (concept illustration)

```
<!-- Template-driven approach (conceptual) -->
<form #form='ngForm' (ngSubmit)='onSubmit(form)'>
  <input name='email' [(ngModel)]='email' required email />
  <input name='password' [(ngModel)]='password' required minlength='6' />
  <button type='submit' [disabled]='form.invalid'>Submit</button>
</form>

<!-- SupplyChainX filter form (reactive equivalent) -->
<form [formGroup]='filters' (ngSubmit)='onFilter()'>
  <select formControlName='locationId'>...</select>
  <select formControlName='itemId'>...</select>
</form>
```

HTML

41

Accessing RESTful API – GET / POST / PUT / DELETE / PATCH

All HTTP methods used in SupplyChainX


```
// GET - fetch data
this.http.get<OrderResponseDto[]>(this.apiUrl, { params })
this.http.get<ItemResponseDto[]>(this.apiUrl)
this.http.get<KpiReportDto>(`${this.apiUrl}/report/kpi`)

// POST - create resource
this.http.post<OrderResponseDto>(this.apiUrl, dto)
this.http.post<void>(`${this.apiUrl}/Login`, loginRequest)

// PUT - full update
this.http.put<UserResponseDto>(`${this.apiUrl}/${id}`, dto)

// PATCH - partial update
this.http.patch<ShipmentResponseDto>(`${this.apiUrl}/${id}`, dto)

// DELETE - remove resource
this.http.delete<void>(`${this.apiUrl}/${id}`)
```


Quick Reference Matrix

The table below maps each concept to the exact file(s) in SupplyChainX where you can find it:

Concept	File(s) in SupplyChainX
Bootstrapping	src/main.ts, src/app/app.config.ts
@Component	Every .component.ts file
selector	app-root (app.ts), app-login, app-header, app-sidebar...
templateUrl vs template	ToastContainerComponent (inline), all others (URL)
Interpolation	header.html (userName), login.html (error), kpi-summary.html
Property Binding	login.html ([type]), [class.is-invalid]), sidebar.html ([routerLink])
Class Binding	inventory.html ([class]=low-stock), kpi-summary.html ([ngClass])
Style Binding	executive-dashboard.html (style=height)
@if / @else	login.html, executive-dashboard.html, inventory.html
@for	sidebar.html (navLinks), inventory.html (rows), create-user.html (roles)
Component Constructor	LoginComponent, SidebarComponent, HeaderComponent
Lifecycle – ngOnInit	HeaderComponent, SidebarComponent, LoginComponent, InventoryComponent
Lifecycle – ngOnDestroy	SidebarComponent, NotificationCenterComponent, AddOrderComponent
Reactive Forms	LoginComponent, AdminCreateUserComponent, WarehouseManageInventoryComponent
Validation	login.html, admin-create-user.html (required, email, minLength)
Data Extraction	LoginComponent (ff.value), WarehouseInventoryComponent (filters.value)
Pipes	executive-kpi-summary.html (number pipe), main.ts (locale/currency)
Creating Services	src/app/services/ – 15+ services
@Injectable	AuthenticationService, ToastService, OrderService...
inject()	AddOrderComponent, NotificationCenterComponent, OrderContextResolver
HTTP Client	OrderService, InventoryService, LoginService, ItemService...
DTOs	src/app/models/ – 20+ interfaces
Interceptors	auth.interceptor.ts, error.interceptor.ts
JWT Decoding	login.service.ts (decodeToken, extractUserFromToken)
JWT Header	auth.interceptor.ts (Authorization: Bearer token)
Browser Storage	authentication.service.ts (localStorage)
Route Definitions	src/app/app.routes.ts
Redirect Route	app.routes.ts (path:"", redirectTo:"admin-dashboard")
Catch-all Route	app.routes.ts (path:"***", redirectTo:"404")
Child Routes	admin-dashboard, planner-dashboard, warehouse-dashboard children
Static Links	sidebar.component.html ([routerLink])
Dynamic Navigation	header.component.ts (router.navigate), login.component.ts
Navigation with Data	app.routes.ts (data:{roles:[...]}), query params (returnUrl, section)
Route Guards	auth.guard.ts (authGuard), role.guard.ts (RoleGuard)
Sibling Communication	OrderContextService (BehaviorSubject), ToastService
Bootstrap Integration	index.html (CDN), all .html templates (Bootstrap classes)
Child Components	ExecutiveDashboard !' KpiSummary, KpiTrends, Risks, Reports
@Input	ExecutiveKpiSummaryComponent (@Input() kpiData), KpiTrends, Risks
Template-Driven Forms	Filter forms in inventory (reactive equivalent)

A

Appendix – SupplyChainX Project Structure

```
src/
% % % main.ts          !• bootstrapApplication()
% % % index.html       !• Bootstrap CDN, <app-root>
% % % app/
%   % % % app.ts        !• Root @Component (selector: app-root)
%   % % % app.html      !• <app-toast-container> + <router-outlet>
%   % % % app.config.ts !• Providers: router, HTTP client, interceptors
%   % % % app.routes.ts !• All route definitions
%   % % % components/
%   %   % % % login/      !• Reactive form, JWT login
%   %   % % % admin-dashboard/ !• Users, Audit Logs, Network (child routes)
%   %   % % % planner-dashboard/ !• Exceptions, Resolution Actions
%   %   % % % logistics-dashboard/ !• Orders, Dispatch, Shipments
%   %   % % % executive-dashboard/ !• KPI, Trends, Risk, Reports (child components)
%   %   % % % procurement-dashboard/ !• Locations, Partners, Orders
%   %   % % % warehouse-dashboard/ !• Inventory, UOM, Items
%   %   % % % shared/      !• Layout (Header, Sidebar), Toast, Notifications
%   % % % services/      !• 15+ injectable services
%   % % % models/        !• 20+ TypeScript DTO interfaces
%   % % % guards/        !• authGuard, RoleGuard, Resolvers
%   % % % interceptors/  !• auth.interceptor, error.interceptor
%   % % % settings/      !• AppSettings (API endpoint)
```

TEXT

Angular Concepts Guide

SupplyChainX Frontend – Complete Reference

All 48+ Angular concepts explained with real code examples from the application